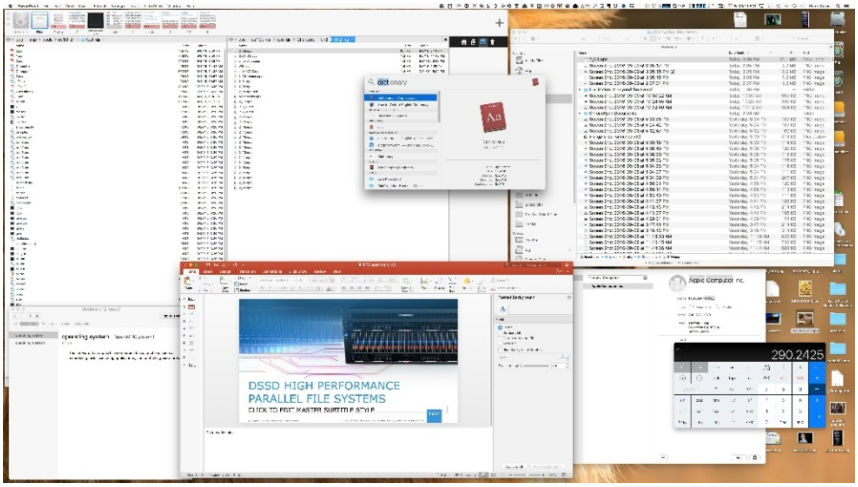


شکل ۲-۴ رابط کاربر گرافیکی macOS.

هرچند که برنامه‌هایی هستند که یک رابط خط‌فرمان برای سیستم‌های موبایل iOS و اندروید فراهم می‌کنند، آن‌ها به‌ندرت استفاده می‌شوند. در عوض، تقریباً تمامی کاربران سیستم‌های موبایل با استفاده از رابط صفحه‌نمایش لمسی با دستگاه‌های خودشان تعامل برقرار می‌کنند.

رابط کاربر می‌تواند از سیستمی به سیستم دیگر و حتی از کاربری به کاربر دیگر در میان یک سیستم فرق کند؛ با این وجود، نوعاً در اصل از ساختار واقعی سیستم برداشته می‌شود. لذا، طراحی یک رابط کاربری مفید و مشهود، یک کارکرد مستقیم سیستم‌عامل نیست. در این کتاب، ما بر روی مسئله‌های اساسی تدارک سرویس کافی برای برنامه‌های کاربر تمرکز می‌کنیم. از نقطه‌نظر سیستم‌عامل، ما بین برنامه‌های کاربر و برنامه‌های سیستم تفاوتی قایل نمی‌شویم.

۲-۳ فراخوانی‌های سیستمی

فراخوانی‌های سیستمی[1] رابطی برای سرویس‌های سیستم‌عامل فراهم می‌کنند. این فراخوانی‌ها عموماً به صورت روال‌هایی به زبان C و C++ هستند، هرچند ممکن است وظایف سطح پایین خاصی (برای مثال، وظایفی که دسترسی مستقیم سخت‌افزار را لازم دارند) به ناچار با استفاده از دستورات زبان اسمبلی هم نوشته شوند.

۲-۳-۱ مثال

پیش از این که به بحث چگونگی تدارک فراخوانی‌های سیستمی توسط سیستم‌عامل وارد شویم، اجازه بدهید تا نحوه‌ی استفاده از فراخوانی‌های سیستمی را با ذکر یک مثال روشن کنیم:

1. System calls

نوشتن برنامه‌ای ساده برای خواندن داده‌ها از یک فایل و کپی آنها در فایل دیگر. اولین ورودی که برنامه لازم دارد اسامی دو فایل مذکور است: فایل ورودی و فایل خروجی. این اسامی بسته به طراحی سیستم‌عامل ممکن است با روش‌های متعددی مشخص شوند. یک رویکرد آن است که اسامی دو فایل را در بخشی از فرمان مشخص کرد—برای مثال، فرمان cp در یونیکس به‌شکل زیر:

```
cp in.txt out.txt
```

این فرمان، فایل ورودی in.txt را به فایل خروجی out.txt کپی می‌کند. رویکرد دوم این است که برنامه از کاربر بخواهد که اسامی را وارد کند. این رویکرد در یک سیستم محاوره‌ای، دنباله‌ای از فراخوانی‌های سیستمی را لازم خواهد داشت؛ اولاً، نمایش یک پیغام در صفحه‌نمایش جهت اطلاع کاربر و در ادامه، خواندن کاراکترهایی از صفحه‌کلید که نام دو فایل را تعریف می‌کنند. در سیستم‌های مبتنی بر ماوس و آیکون، معمولاً منویی از اسامی فایل‌ها در یک پنجره به نمایش در می‌آید. سپس، کاربر می‌تواند نام فایل مبدأ را با استفاده از ماوس انتخاب کرده و ممکن است پنجره‌ای باز شود تا نام فایل مقصد را نیز مشخص کند. این دنباله به فراخوانی‌های سیستمی I/Oای زیادی نیاز دارد.

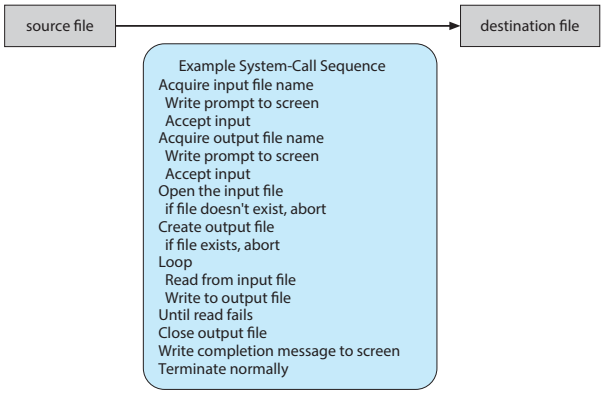
همین که برنامه نام دو فایل را به‌دست آورد، بایستی فایل ورودی را باز کرده و فایل خروجی را ایجاد و باز کند. هر یک از این عملیات، فراخوانی سیستمی دیگری را لازم دارند. بایستی به شرایط خطای احتمالی هر عملیات رسیدگی شود. برای مثال، زمانی که برنامه اقدام به بازکردن فایل ورودی می‌نماید، ممکن است متوجه شود که فایلی با آن نام وجود ندارد یا در مقابل این دسترسی حفاظت می‌شود. در این موارد، برنامه بایستی یک پیغام خطا خروجی نماید (دنباله‌ی فراخوانی سیستمی دیگر) و سپس به طور غیرعادی (فراخوانی سیستمی دیگر) خاتمه یابد. اگر فایل ورودی موجود باشد، در این صورت بایستی یک فایل خروجی جدید ایجاد کنیم. ممکن است متوجه شویم که یک فایل خروجی هم نام با فایل جدید از قبل موجود است. این وضعیت می‌تواند باعث شود که برنامه کنار گذاشته شود[1] (یک فراخوانی سیستمی) یا امکان دارد که فایل موجود را حذف کرده (فراخوانی سیستمی دیگر) و فایل جدیدی را ایجاد کنیم (باز هم فراخوانی سیستمی دیگر). انتخاب دیگر در یک سیستم محاوره‌ای آن است که از کاربر سؤال شود (از طریق دنباله‌ای از فراخوانی‌های سیستمی که پیغامی را برای اطلاع کاربر خروجی می‌نماید و پاسخ کاربر از ترمینال خوانده می‌شود) آیا فایل موجود جایگزین شده یا برنامه کنار گذاشته شود.

موقعی که هر دو فایل آماده شدند، وارد حلقه‌ای می‌شویم که از فایل ورودی خوانده (یک فراخوانی سیستمی) و در فایل خروجی می‌نویسد (فراخوانی سیستمی دیگر). هر یک از دستورات خواندن و نوشتن، در رابطه با شرایط مختلف احتمال خطا بایستی اطلاعات وضعیتی را هم برگردانند. ممکن است برنامه در ورودی متوجه شود که به انتهای فایل رسیده است یا یک خطای سخت‌افزاری در خواندن پیش بیاید (مثل خطای توازن). عملیات نوشتن ممکن است بسته به

1. Abort

دستگاه خروجی (برای مثال، نداشتن فضای کافی در دسترس دیسک) با خطاهای گوناگونی مواجه شود.

سرانجام، بعد از این که کل فایل کپی شد، برنامه می‌تواند هر دو فایل را ببندد (فراخوانی سیستمی دیگر)، پیغامی در کنسول یا پنجره بنویسد (فراخوانی‌های سیستمی بیشتر) و بالاخره به طور عادی خاتمه یابد (آخرین فراخوانی سیستمی). این دنباله‌ی فراخوانی سیستمی در شکل ۲–۵ نشان داده می‌شود.



شکل ۲–۵ مثالی از چگونگی استفاده از فراخوانی‌های سیستمی.

۲-۳-۲ رابط برنامه‌نویسی کاربردی

همان‌گونه که می‌بینید، حتی برنامه‌های ساده نیز ممکن است استفاده‌ی سنگینی از سیستم‌عامل داشته باشند. سیستم‌ها به کرات، هزاران فراخوانی سیستمی را در هر ثانیه اجرا می‌کنند. با این وجود، اکثر برنامه‌نویسان هرگز جزئیات را تا این سطح نمی‌بینند. نویسندگان برنامه‌های کاربردی نوعاً برنامه‌ها را مطابق با یک **رابط برنامه‌نویسی کاربردی**[1] (**API**) طراحی می‌کنند. API، مجموعه‌ای از توابع را که در دسترس یک برنامه‌نویس کاربردی است به علاوه پارامترهای ارسالی به هر یک از توابع و مقادیر برگشتی مورد انتظار برنامه‌نویس را توصیف می‌کند. سه نمونه از رایج‌ترین APIها که در اختیار برنامه‌نویسان کاربردی قرار دارند عبارتند از: API ویندوز برای سیستم‌های ویندوز، API پوزیکس برای سیستم‌های مبتنی بر پوزیکس (که تقریباً تمامی نسخه‌های یونیکس، لینوکس و macOS را شامل می‌شود) و API جاوا برای برنامه‌های اجرایی بر روی ماشین مجازی جاوا. برنامه‌نویس، API را از طریق یک کتابخانه‌ی کد که توسط سیستم‌عامل فراهم می‌شود، دسترسی می‌نماید. این کتابخانه برای برنامه‌های نوشته شده به زبان C در یونیکس و لینوکس، **libc** می‌باشد. توجه داشته باشید —مگر این که مشخصاً عنوان شود—که نام فراخوانی‌های سیستمی به کار رفته در این کتاب نمونه‌های کلی هستند. هر سیستم‌عامل برای هر فراخوانی سیستمی، نام خاص خود را دارد.

1. Application programming interface

در پشت صحنه، توابعی که یک API را می‌سازند فراخوانی‌های سیستمی واقعی را نوعاً از طرف برنامه‌نویس کاربردی احضار می‌کنند. برای مثال، تابع ()CreateProcess در ویندوز (که به شیوه‌ای نه چندان مطلوب برای ایجاد پردازه‌ی جدید استفاده می‌شود) در حقیقت فراخوانی سیستمی ()NTCreateProcess را از هسته‌ی ویندوز احضار می‌کند.

چرا برنامه‌نویس کاربردی به جای این که فراخوانی‌های سیستمی واقعی را احضار کند، برنامه‌نویسی با API را ترجیح می‌دهد؟ دلایل متعددی برای انجام این کار وجود دارد. یک نفع به قابلیت حمل برنامه مربوط می‌شود. یک برنامه‌نویس کاربردی که برنامه‌ای را با API طراحی می‌کند می‌تواند این انتظار را داشته باشد که برنامه‌اش در هر سیستمی که همان API را پشتیبانی کند، کامپایل و اجرا خواهد شد (اگرچه اختلافات معماری به واقع این کار را با مشکلات بیشتری روبه‌رو می‌کند). به علاوه، فراخوانی‌های سیستمی واقعی غالباً جزئیات بیشتری داشته و کار برنامه‌نویس با آنها دشوارتر از کار با API موجود است. با این حال، غالباً یک همبستگی قوی میان یک تابع موجود در API و فراخوانی سیستمی مرتبط با آن در هسته وجود دارد. در حقیقت، بسیاری از APIهای پوزیکس و ویندوز، شبیه فراخوانی‌های سیستمی بومی هستند که توسط سیستم‌عامل‌های یونیکس، لینوکس و ویندوز فراهم می‌شوند.

مثالی از API استاندارد

به عنوان مثالی از یک API استاندارد، تابع ()read را که در سیستم‌های یونیکس و لینوکس وجود دارد، در نظر بگیرید. API این تابع از صفحه‌ی man با احضار فرمان زیر در خط فرمان حاصل می‌شود:

```
man read
```

توصیفی از این API به صورت زیر است:

```
#include <unistd.h>

ssize_t     read(int fd, void *buf, size_t count)
```

return value — function name — parameters

هر برنامه که از تابع ()read استفاده می‌کند بایستی فایل سرآیند unistd.h را لحاظ کند، چرا که این فایل، انواع داده‌های ssize_t و size_t (در میان چیزهای دیگر) را تعریف می‌کند. پارامترهای ارسالی به ()read به صورت زیر هستند:

- int fd— توصیف‌گر فایلی که قرار است خوانده شود.
- void *buf— بافری که داده‌ها به داخل آن خوانده می‌شوند.
- size_t count— حداکثر تعداد بایت‌هایی که به داخل بافر خوانده می‌شوند.

اگر عملیات خواندن موفقیت‌آمیز باشد، تعداد بایت‌های خوانده شده برگردانده می‌شود. مقدار برگشتی ۰، انتهای فایل را نشان می‌دهد. اگر عملیات انجام گرفته خطا داشته باشد، ()read مقدار ۱- بر می‌گرداند.

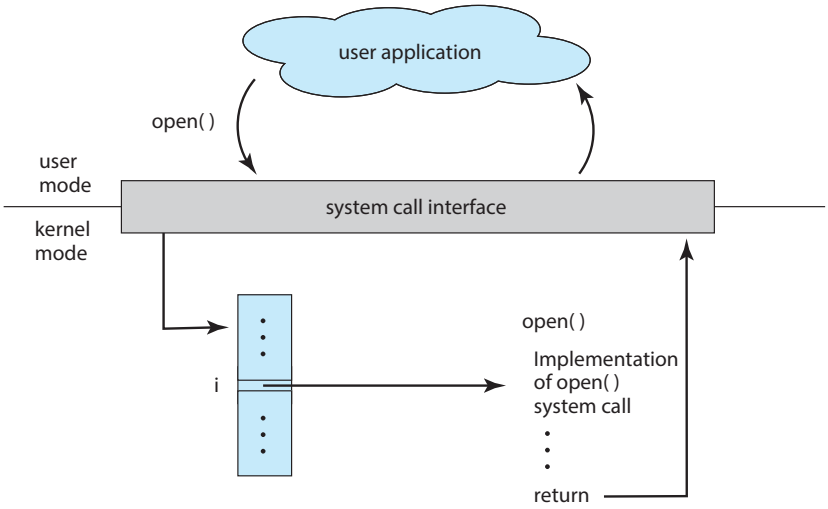
عامل مهم دیگر در رسیدگی به فراخوانی‌های سیستمی، **محیط زمان‌اجرا**[1] (**RTE**) می‌باشد— مجموعه‌ی کاملی از نرم‌افزار مورد نیاز برای اجرای برنامه‌های کاربردی که به یک زبان برنامه‌نویسی مشخصی نوشته شده‌اند، اعم از کامپایلرها یا مفسرها در کنار نرم‌افزارهای دیگر، همچون کتابخانه‌ها و بارگذارها. RTE یک رابط فراخوانی سیستمی فراهم می‌کند که به‌عنوان رابطی به فراخوانی‌های سیستمی فراهم شده توسط سیستم‌عامل عمل می‌کند. رابط فراخوانی سیستم، فراخوانی‌های توابع API را گرفته و فراخوانی سیستمی لازم را از سیستم‌عامل احضار می‌کند. نوعاً، به هر یک از فراخوانی‌های سیستمی شماره‌ای منتسب می‌شود و رابط فراخوانی سیستم، جدولی را نگه می‌دارد که با این شماره‌ها ایندکس شده است. سپس، رابط فراخوانی سیستم، فراخوانی سیستمی خواسته شده را از هسته‌ی سیستم‌عامل احضار و وضعیت فراخوانی سیستمی را بر می‌گرداند.

فراخواننده نیازی ندارد که چیزی در رابطه با چگونگی پیاده‌سازی فراخوانی سیستم یا عملیات آن در طی اجرا بداند. در عوض، فراخواننده فقط لازم دارد که مطیع API بوده و بداند که سیستم‌عامل در پی اجرای آن فراخوانی سیستمی چه کار خواهد کرد. بنابراین، بیشتر جزئیات رابط سیستم‌عامل توسط API از برنامه‌نویس پنهان مانده و توسط RTE اداره می‌شوند. رابطه‌ی میان یک API، رابط فراخوانی سیستم و سیستم‌عامل در شکل ۲-۶ نشان داده می‌شود که چگونگی راه‌اندازی فراخوانی سیستمی ()open را توسط سیستم‌عامل به دنبال درخواست برنامه‌ی کاربر روشن می‌سازد.

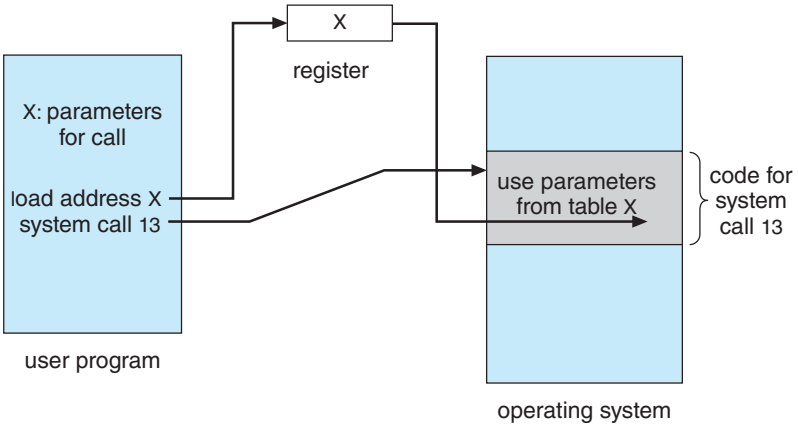
فراخوانی‌های سیستمی، بسته به کامپیوتر مورد استفاده به شیوه‌های متفاوتی اتفاق می‌افتند. اغلب، اطلاعات بیشتری از شناسه‌ی ساده‌ی فراخوانی سیستمی مطلوب مورد نیاز است. نوع و مقدار دقیق اطلاعات بر اساس سیستم‌عامل و فراخوانی خاص تغییر می‌کند. برای مثال، برای دریافت ورودی ممکن است لازم باشد که فایل یا دستگاه مورد استفاده برای مبدأ را در کنار آدرس و طول بافر حافظه‌ای که بایستی ورودی به آن خوانده شود، مشخص کنیم. البته، دستگاه یا فایل و طول بافر ممکن است در فراخوانی به صورت ضمنی باشند.

سه رویکرد کلی برای ارسال پارامترها به سیستم‌عامل استفاده می‌شوند. ساده‌ترین رویکرد، ارسال پارامترها از طریق ثبات‌هاست. با این وجود، در برخی موارد ممکن است پارامترهای بیشتری از تعداد ثبات‌ها مورد نیاز باشند. در اینجا، پارامترها به طور کلی در یک بلاک یا جدولی در داخل حافظه ذخیره می‌شوند و آدرس بلاک به عنوان یک پارامتر از طریق یک ثبات ارسال می‌شود (شکل ۲-۷). لینوکس از ترکیبی از این رویکردها استفاده می‌کند. اگر تعداد پارامترها، پنج یا کمتر باشد، ثبات استفاده می‌شود. اگر تعداد پارامترها بیشتر از پنج پارامتر باشد، روش بلاکی استفاده می‌شود. پارامترها می‌توانند روی **پشته** هم گذاشته شوند تا سیستم‌عامل آنها را از پشته بر دارد. برخی سیستم‌عامل‌ها روش‌های بلاک یا پشته را ترجیح می‌دهند، چون این روش‌ها، تعداد یا طول پارامترهای ارسالی را محدود نمی‌کنند.

1. Run-time environment (RTE)



شکل ۲-۶ رسیدگی به یک برنامه‌ی کاربر که فراخوانی سیستمی open() را احضار کرده است.



شکل ۲-۷ ارسال پارامترها به صورت یک جدول.

۲-۳-۳ انواع فراخوانی‌های سیستمی

فراخوانی‌های سیستمی را می‌توان در شش دسته‌ی اصلی طبقه‌بندی کرد که عبارتند از: **کنترل پردازه، مدیریت فایل، مدیریت دستگاه، نگه‌داری اطلاعات، ارتباطات و حفاظت**. ما در زیر انواع فراخوانی‌های سیستمی را که سیستم‌عامل می‌تواند فراهم کند به‌طور خلاصه بحث می‌کنیم. اکثر این فراخوانی‌های سیستمی، مفاهیم و توابعی را پشتیبانی می‌کنند، یا با آنها پشتیبانی می‌شوند که در فصول آتی مورد بحث قرار می‌گیرند. شکل ۲-۸، انواع فراخوانی‌های سیستمی را که قاعدتاً توسط سیستم‌عامل فراهم می‌شوند خلاصه می‌کند. همان‌گونه که در این کتاب یادآور شدیم، برای ارجاع به فراخوانی‌های سیستمی از نام‌های عمومی استفاده می‌کنیم. با این همه،

مثال‌های واقعی فرخوانی‌های سیستمی سیستم‌های یونیکس، لینوکس و ویندوز را در کل کتاب فراهم می‌کنیم.

- **Process control**
  - load, execute
  - create process, terminate process
  - get process attributes, set process attributes
  - wait event, signal event
  - allocate and free memory
- **File management**
  - create file, delete file
  - open, close
  - read, write, reposition
  - get file attributes, set file attributes
- **Device management**
  - request device, release device
  - read, write, reposition
  - get device attributes, set device attributes
  - logically attach or detach devices
- **Information maintenance**
  - get time or date, set time or date
  - get system data, set system data
  - get process, file, or device attributes
  - set process, file, or device attributes
- **Communications**
  - create, delete communication connection
  - send, receive messages
  - transfer status information
  - attach or detach remote devices
- **Protection**
  - get file permissions
  - set file permissions

- **کنترل پردازه**
  - ایجاد پردازه، خاتمه‌ی پردازه
  - بارگذاری، اجرا
  - خواندن صفات پردازه، انتخاب صفات پردازه
  - رویداد wait، رویداد signal
  - تخصیص و رهایی حافظه
- **مدیریت فایل**
  - ایجاد فایل، حذف فایل
  - بازکردن، بستن
  - خواندن، نوشتن، تغییر مکان
  - خواندن صفات فایل، انتخاب صفات فایل
- **مدیریت دستگاه**
  - درخواست دستگاه، رهایی دستگاه
  - خواندن، نوشتن، تغییر مکان
  - خواندن صفات دستگاه، انتخاب صفات دستگاه
  - اتصال یا قطع منطقی دستگاه‌ها
- **نگهداری اطلاعات**
  - خواندن ساعت یا تاریخ، انتخاب ساعت یا تاریخ
  - خواندن داده‌های سیستمی، انتخاب داده‌های سیستمی
  - خواندن صفات پردازه، فایل یا دستگاه
  - انتخاب صفات پردازه، فایل یا دستگاه
- **ارتباطات**
  - ایجاد، حذف اتصال ارتباطی
  - ارسال، دریافت پیغام‌ها
  - انتقال اطلاعات وضعیتی
  - اتصال یا قطع دستگاه‌های دور
- **حفاظت**
  - خواندن مجوزهای فایل
  - تنظیم مجوزهای فایل

شکل ۲-۸ انواع فراخوانی‌های سیستمی.

مثال‌هایی از فراخوانی‌های سیستمی ویندوز و یونیکس

در زیر، فهرستی از فراخوانی‌های سیستمی مختلف سیستم‌عامل‌های ویندوز و یونیکس نشان داده می‌شوند.

	Windows	Unix
کنترل پردازه	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
مدیریت فایل	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
مدیریت دستگاه	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
نگهداری اطلاعات	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
ارتباطات	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shm_open() mmap()
حفاظت	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

۲-۳-۳-۱ کنترل پردازه

پردازه یا کاری که برنامه‌ای را اجرا می‌کند، ممکن است بخواهد برنامه‌ی دیگری را بارگذاری و اجرا نماید. این امکان، مفسر فرمان را قادر می‌سازد که یک برنامه را برای مثال، با هدایت فرمان کاربر، کلیک ماوس یا فرمان دسته‌ای اجرا کند. پرسش جالب این است که بعد از خاتمه‌ی برنامه‌ی بار شده، کنترل به کجا برمی‌گردد. جواب این سؤال به این مسأله مربوط می‌شود که آیا برنامه‌ی موجود رها می‌شود یا ذخیره می‌گردد و یا همروند با برنامه‌ی جدید به اجرای خود ادامه می‌دهد.

یک برنامه‌ی در حال اجرا نیاز دارد که بتواند اجرایش را به‌صورت عادی (end()) و یا غیرعادی (abort()) خاتمه دهد. اگر یک فراخوانی سیستمی برای ختم عادی برنامه‌ی در حال اجرای فعلی انجام شود، یا اگر برنامه با یک مسئله مواجه شده و یک خطای تله به‌بار بیاورد، بعضاً یک رونوشتی از حافظه گرفته شده و یک پیغام خطا تولید می‌شود. رونوشت در یک فایل سابقه‌ی خاصی در روی دیسک نوشته می‌شود و می‌تواند توسط یک **عیب‌یاب**[1]—برنامه‌ای سیستمی که برای کمک به برنامه‌نویس در یافتن و تصحیح خطاها یا باگ‌ها طراحی شده است— مورد ارزیابی قرار بگیرد تا علت مسئله تعیین شود. تحت شرایط عادی و یا غیرعادی، سیستم‌عامل بایستی کنترل را به مفسر فرمان احضارکننده انتقال دهد. مفسر فرمان در ادامه، فرمان بعدی را می‌خواند. در یک سیستم محاوره‌ای، مفسر فرمان به‌سادگی کارش را با فرمان

1. Debugger

بعدی تداوم می‌بخشد؛ فرض می‌شود که کاربر، یک فرمان مناسبی در پاسخ به هرگونه خطایی صادر خواهد کرد. در یک سیستم GUI، یک پنجره‌ی pop-up ممکن است خطا را به کاربر هشدار بدهد و درخواست راهنمایی کند. برخی سیستم‌ها ممکن است در صورت رخداد خطا امکان عملیات ترمیم به‌خصوصی را فراهم کنند. اگر برنامه خطایی را در ورودیش کشف کند و بخواهد به‌صورت غیرعادی خاتمه یابد، ممکن است بخواهد یک سطح خطا نیز تعریف کند. خطاهای جدی‌تر می‌توانند با پارامتر خطای سطح بالاتر نشان داده شوند. پس می‌توان ختم عادی و غیرعادی را توسط تعریف یک ختم عادی در خطای سطح ۰ ترکیب کرد. مفسر فرمان یا برنامه‌ی بعدی می‌تواند از این سطح خطا به‌صورت خودکار برای تعیین عمل بعدی استفاده کند.

پردازه‌ای که یک برنامه را اجرا می‌کند ممکن است بخواهد برنامه‌ی دیگری را load() و execute() کند. این امکان به مفسر فرمان اجازه می‌دهد تا یک برنامه را تحت هدایت، برای مثال، یک فرمان کاربر یا یک کلیک ماوس اجرا نماید. یک پرسش جالب این است موقعی که برنامه‌ی بارگذاری شده خاتمه پیدا کرد کنترل به کجا برگردانده شود. این پرسش به این موضوع مربوط می‌شود که آیا برنامه‌ی موجود از بین می‌رود، ذخیره می‌شود، یا اجازه می‌یابد تا به‌صورت همروند با برنامه‌ی جدید تداوم اجرا داشته باشد.

اگر بعد از اجرای برنامه‌ی جدید، کنترل به برنامه‌ی موجود بر می‌گردد، بایستی تصویر حافظه‌ی برنامه‌ی موجود را ذخیره کنیم؛ بنابراین، به صورت کارآمد، راهکاری برای یک برنامه ایجاد کرده‌ایم که می‌تواند برنامه‌ی دیگر را فراخوانی کند. اگر برنامه‌ها توأمان اجرای همروند داشته باشند، یک پردازه‌ی جدیدی ایجاد کرده‌ایم که چندبرنامگی دارد. اغلب، یک فراخوانی سیستمی مخصوص این کار وجود دارد (create_process()).

اگر پردازه‌ی جدیدی، یا حتی مجموعه‌ای از پردازه‌هایی را ایجاد کنیم، بایستی قادر باشیم اجرای آن را کنترل کنیم. لازمه‌ی این کنترل آن است که بتوان صفات یک پردازه، اعم از الویت پردازه، حداکثر زمان اجرای مجاز و از این دست را تعیین کرد (get_process_attributes() و set_process_attributes()). همچنین ممکن است بخواهیم پردازه‌ای را که ایجاد کرده‌ایم خاتمه بدهیم (terminate_process())، در صورتی‌که متوجه شویم نادرست بوده یا دیگر به آن نیاز نداریم.

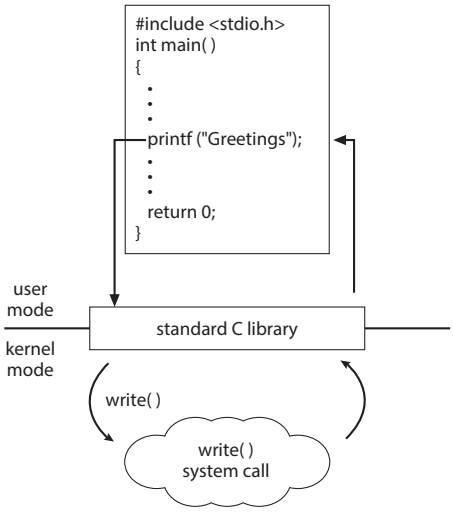
با ایجاد پردازه‌های جدید امکان دارد که لازم باشد به انتظار آنها بمانیم تا اجرای خود را به اتمام برسانند. ممکن است بخواهیم مدت زمان معینی منتظر بمانیم (wait_time()). به احتمال بیشتر هم ممکن است بخواهیم منتظر رخداد رویداد خاصی بمانیم (wait_event()). بنابراین، بایستی پردازه‌ها، رخداد آن رویداد را متوجه شوند (signal_event()).

زیاد اتفاق می‌افتد که دو یا چند پردازه می‌خواهند داده‌های مشترکی داشته باشند. سیستم‌عامل‌ها اغلب برای تضمین جامعیت داده‌های مشترک، فراخوانی‌های سیستمی فراهم می‌کنند که پردازه بتواند داده‌های مشترک را قفل نماید. در ادامه، هیچ پردازه‌ی دیگری نمی‌تواند داده‌ها را دستیابی کند تا این که قفل رها شود. این قبیل فراخوانی‌های سیستمی نوعاً شامل acquire_lock() (کسب قفل) و realease_lock() (رهاسازی قفل) هستند. این نوع فراخوانی‌های

سیستمی که به هماهنگی پردازه‌های همروند مربوط می‌شوند با جزئیات بیشتر در فصل ۶ و فصل ۷ مورد بحث قرار می‌گیرند.

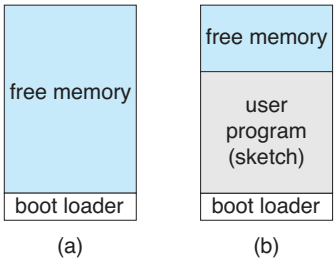
کتابخانه‌ی استاندارد C

کتابخانه‌ی استاندارد C، بخشی از رابط فراخوانی سیستمی بسیاری از نسخه‌های یونیکس و لینوکس را فراهم می‌کند. به عنوان یک مثال، برنامه‌ای به زبان C داریم که دستور printf() را احضار می‌کند. کتابخانه‌ی C، این فراخوانی را گرفته و فراخوانی (یا فراخوانی‌های) سیستمی لازم—در اینجا، فراخوانی سیستمی write()—را از سیستم‌عامل احضار می‌کند. کتابخانه‌ی C، مقدار برگشتی فراخوانی سیستمی write() را می‌گیرد و دوباره به برنامه‌ی کاربر بر می‌گرداند:



تنوع و اَشکال کنترل پردازه چنان زیاد است که در ادامه، از دو مثال—یکی مربوط به یک سیستم تک‌وظیفه و دیگری یک سیستم چندوظیفگی—برای روشن‌شدن این مفاهیم استفاده می‌کنیم. Arduino یک سکوی سخت‌افزاری ساده‌ای شامل یک میکروکنترلر همراه با حس‌گرهای ورودی است که به انواع رویدادها پاسخ می‌دهد؛ تغییرات نور، درجه حرارت، و فشار بارومتری تنها نمونه‌هایی از این رویدادها می‌باشند. ما برای نوشتن یک برنامه برای Arduino ابتدا برنامه را روی یک PC می‌نویسیم و سپس برنامه‌ی کامپایل شده (معروف به **sketch**) را از PC به حافظه‌ی فلش Arduino از طریق یک اتصال USB انتقال می‌دهیم. سکوی استاندارد Arduino یک سیستم‌عامل فراهم نمی‌کند؛ در عوض، یک تکه‌ی نرم‌افزار کوچکی تحت عنوان **بارگذار بوت** sketch را به ناحیه‌ی خاصی در حافظه‌ی Arduino بارگذاری می‌کند (شکل ۲-۹). همین که sketch بارگذاری شد، اجرایش را به انتظار رویدادهایی که برنامه‌نویسی شده است تا پاسخ بدهد، شروع می‌کند. برای مثال، اگر حس‌گر درجه حرارت Arduino تشخیص بدهد که درجه حرارت از یک آستانه‌ی خاصی فراتر رفته است، sketch ممکن است Arduino را وادار کند تا موتور یک فَن را

روشن کند. Arduino یک سیستم تک‌وظیفه‌ای محسوب می‌شود، چرا که تنها یک sketch می‌تواند در هر لحظه در حافظه حضور داشته باشد؛ اگر sketch دیگری بارگذاری شود، جانشین sketch موجود می‌شود. به‌علاوه، Arduino به‌غیر از حس‌گرهای ورودی سخت‌افزاری، هیچ‌گونه رابط کاربری فراهم نمی‌کند.



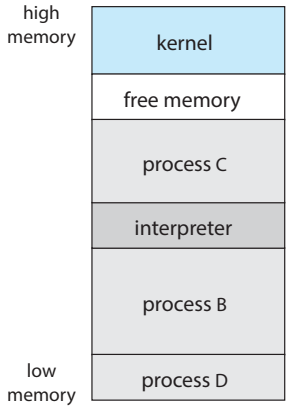
شکل ۲-۹ اجرای Arduino. (a) موقع بالاآمدن سیستم. (b) اجرای یک sketch.

FreeBSD (مشتق از یونیکس برکلی[1]) مثالی از یک سیستم چندوظیفگی است. زمانی که کاربری به سیستم وارد می‌شود، پوسته‌ی انتخابی کاربر، به‌انتظار فرمان‌ها و برنامه‌های اجرایی درخواستی کاربر، اجرا می‌شود. با این وجود، از آن جایی که FreeBSD یک سیستم چندوظیفه است، مفسر فرمان می‌تواند اجرایش را در حالی که برنامه‌ی دیگری اجرا می‌شود، ادامه بدهد (شکل ۲-۱۰). برای شروع یک پردازه‌ی جدید، پوسته یک فراخوانی سیستمی ()fork اجرا می‌کند. سپس برنامه‌ی انتخابی، توسط فراخوانی سیستمی ()exec به داخل حافظه بار شده و اجرا می‌شود. بسته به چگونگی صدور فرمان، پوسته پس از آن، یا به انتظار اتمام پردازه می‌ماند و یا پردازه را «در پس‌زمینه» اجرا می‌کند. در حالت دوم، پوسته بلافاصله منتظر ورود فرمان دیگری می‌ماند. زمانی که یک پردازه در پس‌زمینه اجرا می‌شود، نمی‌تواند مستقیماً از صفحه‌کلید ورودی دریافت کند، چون پوسته از این منبع استفاده می‌کند. بنابراین، I/O از طریق فایل‌ها یا یک رابط GUI انجام می‌شود. در این میان، کاربر آزاد است که از پوسته بخواهد برنامه‌های دیگری را اجرا کند، ناظر پیشروی پردازه‌ی در حال اجرا باشد، الویت آن برنامه را تغییر بدهد و کارهایی از این قبیل. زمانی که پردازه انجام پذیرفت، فراخوانی سیستمی ()exit را اجرا می‌کند تا خاتمه یابد و به پردازه‌ی احضارکننده، کد وضعیت ۰ یا کد خطای غیر صفر را بر می‌گرداند. سپس، این کد وضعیت یا خطا در اختیار پوسته یا سایر برنامه‌ها قرار می‌گیرد. پردازه‌ها در فصل ۳ با یک برنامه‌ی نمونه که از فراخوانی‌های سیستمی ()fork و ()exec استفاده می‌کند، مورد بحث واقع می‌شوند.

۲-۳-۳-۲ مدیریت فایل

سیستم‌فایل با جزئیات بیشتری در فصل‌های ۱۳ تا ۱۵ مورد بحث قرار می‌گیرد. در اینجا، چندین فراخوانی سیستمی رایج را در ارتباط با فایل‌ها مشخص می‌کنیم.

1. Berkeley UNIX



شکل ۲-۱۰ FreeBSD در حال اجرای برنامه‌های متعدد.

در ابتدا لازم داریم که بتوانیم فایل‌ها را ()create و ()delete کنیم. هر یک از این فراخوانی‌های سیستمی، نام فایل و احتمالاً برخی صفات فایل را لازم خواهند داشت. همین که فایل ایجاد می‌شود، نیاز داریم که آن را باز کرده و از آن استفاده کنیم. همین‌طور ممکن است بخواهیم فایل را خوانده (()read)، نوشته (()write) یا تغییر موقعیت (()reposition) (برای مثال، پرش به انتها یا برگشت به ابتدای فایل) بدهیم. بالاخره، نیاز داریم که فایل را ببندیم (()close) تا نشان بدهیم که دیگر از آن استفاده نمی‌کنیم.

ممکن است در دایرکتوری‌ها نیز به این مجموعه عملیات نیاز پیدا کنیم، در صورتی که یک ساختار دایرکتوری هم برای سازماندهی فایل‌ها در سیستم‌فایل داشته باشیم. به علاوه، برای فایل‌ها و یا دایرکتوری‌ها نیاز داریم که بتوانیم مقادیر صفات مختلف را تعیین کرده و احتمالاً در صورت لزوم آنها را دومرتبه انتخاب کنیم. صفات فایل، شامل نام فایل، نوع فایل، کدهای حفاظت، اطلاعات حسابداری و از این دست هستند. برای انجام این کار، حداقل دو فراخوانی سیستمی ()get_file_attributes و ()set_file_attributes لازمند. بعضی سیستم‌عامل‌ها، فراخوانی‌های سیستمی بیشتری همچون فراخوانی‌های ()move و ()copy فایل‌ها را نیز فراهم می‌کنند. سیستم‌های دیگر ممکن است یک API داشته باشند که این عملیات را با استفاده از کد و فراخوانی‌های سیستمی دیگر انجام بدهند و برخی سیستم‌ها نیز امکان دارد فقط برنامه‌های سیستمی انجام این وظایف را فراهم کنند. اگر برنامه‌های سیستمی بتوانند توسط سایر برنامه‌ها فراخوانی شوند، در این صورت برای برنامه‌های دیگر یک API محسوب می‌شوند.

۲-۳-۳-۳ مدیریت دستگاه

یک پردازه برای اجرا ممکن است به منابع متعددی—حافظه‌ی اصلی، درایوهای دیسک، دسترسی فایل‌ها و از این دست— نیاز داشته باشد. اگر منابع در دسترس باشند، می‌توان آنها را اعطا کرد و کنترل را به پردازه‌ی کاربر برگرداند. در غیر این صورت، پردازه به ناچار منتظر می‌ماند تا این که منابع کافی مورد نیازش در دسترس قرار بگیرند.

منابع مختلفی را که توسط سیستم‌عامل کنترل می‌شوند می‌توان به عنوان دستگاه تصور کرد. برخی از این دستگاه‌ها فیزیکی (برای مثال، درایوهای دیسک) هستند، حال آنکه دستگاه‌های دیگری هستند که می‌توان آنها را دستگاه‌های انتزاعی یا مجازی (برای مثال، فایل‌ها) به حساب آورد. اگر سیستم کاربران متعددی داشته باشد، برای حصول اطمینان از استفاده‌ی انحصاری دستگاه‌ها ممکن است ملزم باشیم که ابتدا دستگاه را درخواست (request()) کنیم. بعد از این که کار خود را با دستگاه تمام کردیم بایستی آن را رها (release()) کنیم. این توابع، مشابه فراخوانی‌های سیستمی open() و close() در فایل‌ها هستند. سیستم‌عامل‌های دیگری هستند که دسترسی دستگاه‌ها را بدون اعمال مدیریت فراهم می‌کنند. مشکلی که این سیستم‌ها خواهند داشت، رقابت بالقوه برای دستگاه‌ها و احتمالاً بن‌بست خواهد بود که در فصل ۸ مطرح می‌شود.

همین که دستگاه را درخواست کردیم (و در اختیار ما قرار گرفت)، می‌توانیم آن دستگاه را درست همانند فایل‌ها، خوانده (read())، نوشته (write()) و (احتمالاً) در میان آن تغییر موقعیت دهیم (reposition()). در حقیقت، شباهت میان دستگاه‌های I/O و فایل‌ها آن قدر زیاد است که بسیاری از سیستم‌عامل‌ها، از جمله یونیکس، آنها را به صورت ساختار ترکیبی فایل-دستگاه با هم ادغام می‌کنند. در اینجا، مجموعه‌ای از فراخوانی‌های سیستمی، توأمان برای فایل‌ها و دستگاه‌ها استفاده می‌شوند. بعضاً، دستگاه‌های I/O با اسامی فایل خاص، گذاشتن دایرکتوری یا صفات فایل ویژه‌ای شناسایی می‌شوند.

رابط کاربر نیز می‌تواند ظاهر فایل‌ها و دستگاه‌ها را مشابه همدیگر نمایش بدهد، هرچند که فراخوانی‌های سیستمی زیرین آنها شباهتی به هم ندارند. این هم نمونه‌ی دیگری از تصمیم‌گیری‌های طراحی فراوانی است که در ساخت یک سیستم‌عامل و رابط کاربر آن مطرح می‌شود.

۲-۳-۳-۴ نگه‌داری اطلاعات

بسیاری از فراخوانی‌های سیستمی، خیلی ساده با هدف انتقال اطلاعات میان برنامه‌ی کاربر و سیستم‌عامل به وجود آمده‌اند. برای نمونه، اکثر سیستم‌ها یک فراخوانی سیستمی برای اعلام ساعت (time()) و تاریخ جاری دارند (date()). فراخوانی‌های سیستمی دیگری می‌توانند اطلاعاتی در ارتباط با سیستم، اعم از شماره‌ی نسخه‌ی سیستم‌عامل، حجم آزاد فضای حافظه یا دیسک و از این قبیل برگردانند.

مجموعه‌ی دیگری از فراخوانی‌های سیستمی هستند که در عیب‌یابی یک برنامه به کار می‌آیند. بسیاری از سیستم‌ها برای رونوشت از حافظه فراخوانی‌های سیستمی دارند (dump()). این تمهیدات برای عیب‌یابی سودمند هستند. برنامه‌ی strace که در سیستم‌های لینوکس در دسترس می‌باشد، هر فراخوانی سیستمی را که اجرا می‌شود، لیست می‌کند. حتی ریزپردازنده‌ها، یک مد CPU تحت عنوان **تک‌گام** دارند که در آن، بعد از هر دستورالعمل یک تله توسط CPU اجرا می‌شود. تله معمولاً توسط عیب‌یاب گرفته می‌شود.

بسیاری از سیستم‌عامل‌ها، یک سابقه‌ی زمانی هم از یک برنامه تهیه می‌کنند که مدت زمان اجرای برنامه را در یک مکان خاص یا مجموعه‌ای از مکان‌ها نشان می‌دهد. سابقه‌ی زمانی به یک امکان ردیابی یا وقفه‌های منظم تایمر نیازمند است. با هر بار رخداد وقفه، مقدار شمارنده‌ی برنامه

ثبت می‌شود. اگر وقوع وقفه‌ها تناوب کافی داشته باشد، یک تصویر آماری از زمان صرف‌شده در بخش‌های مختلف برنامه به‌دست می‌آید.

به علاوه، سیستم‌عامل اطلاعاتی را در رابطه با تمامی پردازه‌هایش نگه می‌دارد که برای دسترسی به این اطلاعات از فراخوانی‌های سیستمی استفاده می‌شود. فراخوانی‌ها عموماً برای خواندن و تنظیم اطلاعات مربوط به پردازه نیز استفاده می‌شوند (get_process_attributes() و set_process_attributes()). در بخش ۳-۱-۳، در رابطه با این که چه اطلاعاتی در اصل نگه‌داری می‌شوند، بحث می‌کنیم.

۲-۳-۳-۵ ارتباط

دو مدل ارتباطی رایج میان پردازه‌ها وجود دارند که عبارتند از: مدل عبور پیغام[1] و مدل حافظه‌ی مشترک[2]. در **مدل عبور-پیغام**، پردازه‌های مرتبط جهت انتقال اطلاعات با همدیگر از مبادله‌ی پیغام استفاده می‌کنند. تبادل پیغام میان پردازه‌ها می‌تواند به صورت مستقیم یا از طریق یک صندوق‌پستی مشترک انجام شود. پیش از این که ارتباط برقرار شود، بایستی یک اتصال[3] باز شود. باید نام طرف ارتباط دیگر مشخص شود که آیا پردازه‌ی دیگری از همان سیستم است، یا پردازه‌ی دیگری است که از کامپیوتر دیگری به‌وسیله‌ی شبکه‌ی ارتباطی اتصال دارد. هر کامپیوتر در شبکه یک **نام میزبان** دارد که از روی آن کاملاً قابل شناسایی است. میزبان نیز یک شناسه‌ی شبکه، همچون آدرس IP دارد. به طور مشابه، هر پردازه یک **نام پردازه** دارد و این نام به یک شناسه ترجمه می‌شود که سیستم‌عامل می‌تواند با آن به پردازه رجوع کند. این ترجمه را فراخوانی‌های سیستمی get_hostid() و get_processid() انجام می‌دهند. در ادامه، شناسه‌ها بسته به مدل ارتباطی سیستم، به فراخوانی‌های سیستمی همه‌منظوره‌ی open() و close() که توسط سیستم‌فایل فراهم می‌شوند یا به فراخوانی‌های سیستمی open_connection() و close_connection() ارسال می‌شوند. پردازه‌ی طرف گیرنده معمولاً با فراخوانی accept_connection() اجازه‌ی برقراری ارتباط را صادر می‌کند. اکثر پردازه‌هایی که اتصالات را دریافت می‌کنند *گماشتگان*[4] خاص‌منظوره‌ای هستند که برنامه‌های سیستمی بوده و با همین هدف مهیا می‌شوند. آنها یک فراخوانی انتظارکشی اتصال (wait_for_connection()) اجرا کرده و با برقراری ارتباط بیدار می‌شوند. مبدأ ارتباط، به نام *کارفرما* و گماشته‌ی گیرنده، به نام *کارگزار* معروف است، سپس با فراخوانی‌های read_message() و write_message() به مبادله‌ی پیغام‌ها می‌پردازند. فراخوانی سیستمی close_connection() به ارتباط خاتمه می‌دهد.

در **مدل حافظه‌ی مشترک**، پردازه‌ها برای دسترسی نواحی حافظه‌ای که تحت مالکیت پردازه‌های دیگر هستند از فراخوانی‌های سیستمی shared_memory_create() و shared_memory_attach() استفاده می‌کنند. به یاد دارید که سیستم‌عامل اصولاً سعی دارد از دسترسی حافظه‌ی پردازه،

1. Message-passing model
2. Shared-memory model
3. Connection
4. Daemons

توسط پردازه‌های دیگر جلوگیری کند. برای اشتراک حافظه لازم است که دو یا چند پردازه برای حذف این محدودیت با هم به توافق برسند. به این ترتیب، می‌توانند اطلاعات را با خواندن و نوشتن داده‌ها در این بخش‌های مشترک مبادله کنند. شکل و مکان داده‌ها توسط این پردازه‌ها تعیین می‌شود و تحت کنترل سیستم‌عامل نیست. این پردازه‌ها هستند که باید اطمینان بدهند همزمان یک مکان حافظه را نمی‌نویسند. این راهکارها در فصل ۶ بحث می‌شوند. در فصل ۴، گونه‌ای از طرح پردازه—*نخ‌ها*[1]—را خواهیم دید که مقداری از حافظه را به طور پیش‌فرض به اشتراک می‌گیرند.

هر دو مدلی که در اینجا بحث کردیم در سیستم‌عامل‌ها رایج هستند و بیشتر سیستم‌عامل‌ها هر دو را پیاده‌سازی می‌کنند. عبور پیغام برای تبادل حجم کوچک‌تر داده‌ها مفید است، چون نیازی به اجتناب از برخوردها ندارد. پیاده‌سازی آن نیز برای ارتباط میان کامپیوترها نسبت به حافظه‌ی مشترک ساده‌تر است. ارتباط حافظه‌ی مشترک، حداکثر سرعت را داشته و مناسب است، چرا که می‌تواند در یک کامپیوتر با سرعت‌های حافظه انجام بگیرد. با این وجود، مسائلی هم در حوزه‌های حفاظت و همگامی میان پردازه‌هایی که حافظه را به اشتراک می‌گیرند، وجود دارد.

۲-۳-۳-۶ حفاظت

حفاظت، راهکاری برای کنترل دسترسی منابعی است که توسط سیستم کامپیوتری فراهم می‌شوند. از نظر تاریخی، حفاظت فقط دغدغه‌ی سیستم‌های کامپیوتری چندبرنامگی با کاربران متعدد بود. با این وجود، با ظهور شبکه و اینترنت، تمامی سیستم‌های کامپیوتری از کارگزارها گرفته تا دستگاه‌های دستی موبایل بایستی نگران حفاظت باشند.

نوعاً، فراخوانی‌های سیستمی که حفاظت را فراهم می‌کنند عبارتند از set_permission() و get_permission()، که تنظیمات مجوز منابعی چون فایل‌ها و دیسک‌ها را دستکاری می‌نمایند. فراخوانی‌های allow_user() و deny_user() مشخص می‌کنند که آیا کاربران خاصی می‌توانند —یا نمی‌توانند— به منابع معینی دسترسی داشته باشند. حفاظت را در فصل ۱۷ و ملاحظات امنیتی بیشتر را—که شامل استفاده از حفاظت در مقابل تهدیدات خارجی است—در فصل ۱۶ پوشش می‌دهیم.

۲-۴ سرویس‌های سیستمی

وجه دیگر یک سیستم مدرن، مجموعه‌ی سرویس‌های سیستمی آن است. شکل ۱-۱ را به خاطر بیاورید که سلسله‌مراتب منطقی کامپیوتر را نشان می‌داد. سخت‌افزار در پایین‌ترین سطح قرار دارد. بعد از آن سیستم‌عامل، سپس سرویس‌های سیستمی و در نهایت، برنامه‌های کاربردی قرار می‌گیرند. **سرویس‌های سیستمی** که به **برنامه‌های سودمند سیستمی**[2] نیز معروفند، محیطی ساده و راحت را برای توسعه و اجرای برنامه فراهم می‌کنند. برخی از آنها به عنوان رابط‌های

1. Threads
2. System utilities